

D5: Retrospective & Self-Assessment (Part 2)

Student: 210104004053 | Due: 22 May 2026

Section 1 — Retrospective

What I would change if starting over. The most consequential decision I would revisit is the `weight` type. I defined it in §4.2, §4.5, and the lexer, gave it its own token class with kg and lb suffix recognition, and implemented lb-to-kg conversion in the interpreter — but none of the three D3 test programs ever declares a weight variable. The type is fully specified and correctly rejected by the type checker (weight arithmetic is blocked, type mismatches are caught), yet it goes unexercised in the executed output. If I were starting over I would either include a fourth test program that uses `weight` as a function parameter — something like a function that checks whether a barbell weight exceeds a training threshold — or I would simplify the type to a plain alias for float and invest the saved design space in the `Day.setCount` dot notation, which I specified in §4.3 and §4.5 but never added to the parser. Adding dot notation to `parse_primary` requires roughly ten lines: after consuming a plain `IDENT`, peek at the next token; if it is `TOK_DOT`, consume it, expect the literal `setCount`, and return an `EXPR_IDENT` node with the dot included in `sval`. The type checker already handles this path. Not finishing the parser side means D1 §4.3 describes a construct that is never exercised end-to-end.

What had to be scoped out. The most significant gap is control-flow analysis for `return` statements. The type checker correctly verifies the type of a `return` expression when one is present, and it correctly rejects `return` inside a `routine` body. But it does not detect a `func` that falls off the end without returning — the interpreter silently returns `rtval_int(0)`, which is a runtime type violation for any `func` declared to return `bool`, `float`, or a non-zero-valued `int`. Fixing this would require tracking whether every control-flow path through a `func` body ends in a `return` — a dataflow problem beyond what §4.5 requires and beyond the scope of a two-pass type checker. I documented the limitation in D1 §4.5. The second scoped-out item is dot notation parsing, described above. Both limitations are honest omissions with documented reasons rather than silent failures.

The hardest single design decision. The hardest decision was choosing no implicit coercion at all — not even the standard `int-to-float` widening that Sebesta §7.4 calls the least dangerous coercion in practice. Every other language the examiner could ask me to compare WorkoutLang to — C, Java, Python — allows this widening silently. My justification in §4.5 is that the target user is a fitness coach, not a programmer, and that silent type conversions are a source of bugs that experts can reason about but novices cannot predict. This is defensible in the exam, but the trade-off is real: a user who writes `float result = totalVolume(5, 5)` gets a type error because `totalVolume` returns `int`, even though the assignment is obviously safe. The decision forced me to think more carefully about the return types of every function in the test programs and made the type checker simpler to implement — there are no coercion rules to track — but it also made the language genuinely less convenient to use. I stand by it as the right call for the domain, but it is the one decision I could imagine revisiting if I had real target users to consult.

Section 2 — Self-Assessment

Grades assigned against the rubric in §5 of the project handout.

Dimension	Grade	Justification
Correctness	B	All three valid programs execute end-to-end with correct output; all eight error cases (five parse, three type/runtime) fire on the right line with the right message; gcc produces zero warnings. The grade is not A because <code>Day.setCount</code> dot notation is specified in D1 §4.3 but is absent from the parser, and the <code>weight</code> type has no executed test program — two features are present in the spec but not exercised in D3.
Design Justification	A	Every major decision in D1 §4.5–4.8 names the Sebesta section it draws from, gives a fitness-domain justification specific to <code>WorkoutLang</code> , and states the accepted trade-off explicitly. The no-coercion argument (coaches cannot debug silent type conversions), the name-equivalence argument (duration and weight are structurally identical but semantically distinct), and the no-arithmetic-on-weight argument (<code>kg squared</code> is dimensionally nonsensical) are all original and domain-specific rather than generic boilerplate.
Use of Sebesta Framework	A	Section numbers are cited accurately and consistently throughout: §6.12 for strong typing vs §6.11 for static type checking (distinguished explicitly), §6.14 for name equivalence, §5.4.2 and §5.4.3 for stack-dynamic vs static lifetime, §3.5 for big-step operational semantics, §9.2 for call-by-value. The S-Set rule is in standard big-step form with premises as range/membership checks, not evaluation judgements, which matches Sebesta's treatment of terminal productions.
Originality	B	<code>WorkoutLang</code> has a genuine domain: the <code>workout/day/set</code> hierarchy is not a reskin of any textbook example, and the <code>DURATION_LIT</code> and <code>WEIGHT_LIT</code> token types with physical unit suffixes are domain-specific at the lexical level. The repeat N weeks loop with no iteration counter is a deliberate restriction that has no analogue in the handout examples. The grade is not A because the language is still relatively simple — user-defined types are absent, and the <code>Day</code> record has only one field — which limits how far the type system design questions can go.
Code Quality	A	The five source files (lexer, parser, symbol table, type checker, interpreter) have clean separation of concerns: the symbol table has no knowledge of AST nodes, the type checker has no knowledge of runtime values, and the interpreter has no knowledge of type-checker internals. All cross-cutting concerns (error format, scope management) go through shared APIs. The <code>SYM_FRAME_FULL</code> silent-drop bug was caught and fixed before submission. Zero warnings under <code>gcc -Wall -Wextra -std=c11</code> .
Exam Performance (predicted)	B	I am confident on the core decisions: the S-Set bridging argument, name-equivalence justification, static scoping implementation via environment chain, call-by-value ordering in <code>EXPR_CALL</code> , short-circuit evaluation, and the no-coercion rationale. The likely weak point is the dot notation gap — if asked to show <code>Day.setCount</code> executing end-to-end I cannot, because the parser does not produce the right AST node. I know the fix (ten lines in <code>parse_primary</code>) but the implementation is incomplete.